

AI OPEN SOURCE SOFTWARE (OSS)

Asynchronous Work

Collaborate from Anywhere

비동기 협업의 원칙과 도구를 이해하고
글로벌 개발 환경에서의 효율적인 소통 방식을 학습합니다.

SECTION 01

비동기 협업: 언제 어디서나 함께하기

동시성이 아닌 시간차 기반 협업을 통해
글로벌 팀과의 생산적 협업을 구현합니다.



학습 목표 (Learning Objectives)

이번 주차 강의를 통해 비동기 협업의 핵심 원칙을 이해하고
글로벌 개발 환경에 적용하는 방법을 배웁니다.

학습 목표

이번 세션을 통해 비동기 협업의 핵심 가치와 실천 방법을 습득합니다.



핵심 원칙 이해

비동기 협업이 필요한 이유와
'Default to Async' 등 5가지 핵심
원칙을 이해합니다.



도구 활용 능력

GitHub Discussions, Wiki,
ADR 등 비동기 소통을 돕는 도구
의 효과적 사용법을 익힙니다.



문서 중심 문화

구두 소통의 한계를 넘어서는
'Write Things Down' 문화를 구
축하고 실천합니다.



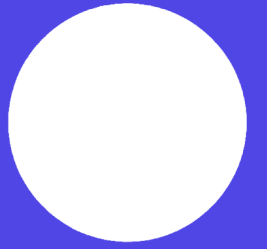
글로벌 협업

다양한 시간대의 팀원들과 효율적
으로 소통하며 시차를 극복하는 방
법을 배웁니다.

SECTION 03

왜 비동기 협업인가?

현대 개발 환경의 변화와
동기 vs 비동기 방식의 장단점을 비교합니다.





| 현대 개발 환경의 변화

물리적 제약이 있는 전통적 환경에서 시공간을 초월한 현대적 분산 협업 환경으로의 전환

전통적 환경

-  같은 사무실 (Co-located)
-  같은 시간대 (Same Timezone)
-  동시 회의 (Sync Meetings)
-  실시간 소통 (Real-time Chat)

SHIFT



| 동기(Synchronous) 커뮤니케이션

실시간 상호작용을 통해 즉각적인 피드백을 주고받는 전통적인 협업 방식입니다.



주요 특징

- 참여자 간의 실시간 대화와 소통
- 질문에 대한 즉각적인 피드백 확인
- 참여자의 높은 집중과 동시 접속 요구



대표 예시

- 화상/대면 회의 (Zoom, Meet)
- 긴급한 전화 통화
- Slack/Teams 실시간 채팅
- 동시 코드 작업 페어 프로그래밍



장점 및 단점

👍 장점

빠른 의사결정, 비언어적 맥락 파악 등 직관적 소통 가능

👎 단점

시간대 제약(Time zone), 업무 흐름 끊김(Interruption), 기록 부족



| 비동기(Asynchronous) 커뮤니케이션

시간과 장소의 제약을 넘어선 효율적인 글로벌 협업 방식



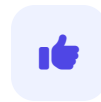
주요 특징

- 시간차를 둔 대화 방식
- 즉답보다 숙고된 응답
- 유연한 참여 가능
- 자발적 기여 문화



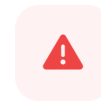
대표 예시

- 이메일 커뮤니케이션
- GitHub 이슈 코멘트
- Pull Request 코드 리뷰
- Wiki 등 문서화 활동



장점 (Pros)

- 업무 유연성 및 몰입도
- 높은 생산성과 기록화
- 글로벌 협업 용이
- 방해 없는 업무 환경



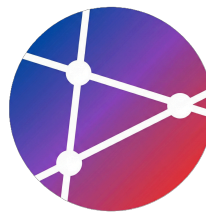
단점 (Cons)

- 느린 피드백 루프
- 텍스트 기반 오해 가능성
- 초기 적응의 어려움
- 사회적 고립감 우려

SECTION 03

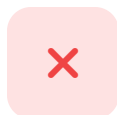
비동기 협업의 핵심 원칙

Default to Async부터 시간대 존중까지,
성공적인 원격 근무를 위한 5가지 불변의 법칙



1. Default to Async (기본은 비동기)

모든 커뮤니케이션에 대해 "이게 꼭 동기(실시간)여야 하나?"라는 질문을 먼저 던져야 합니다.



동기로 하지 말아야 할 것

비동기(Async) 방식 권장

- 단순 정보 공유 (공지사항, 자료 전달)
- 상태 업데이트 (데일리 스크럼, 진척도 보고)
- 코드 리뷰 (집중력이 필요한 상세 검토)
- 일반적인 질문 (답변이 급하지 않은 문의)



동기로 해야 할 것

실시간(Sync) 방식 필요

- ✓ 긴급한 이슈 (서버 장애, 보안 사고 대응)
- ✓ 복잡한 의사결정 (여러 이해관계자 조율 필요)
- ✓ 브레인스토밍 (창의적 아이디어 발산 단계)
- ✓ 팀 빌딩 (정서적 유대감 형성 및 친목)



2. Write Things Down

기록하지 않은 지식은 사라집니다. 모든 것을 문서화하여 팀의 자산으로 만드세요.



구두 (Verbal)

순간적이고 사라짐
(Volatile)



RECOMMENDED

문서 (Written)

영구적이고 검색 가능
(Permanent)

✔ 반드시 문서화해야 할 항목들



결정 사항 및 배경

Why에 대한 기록 (ADR)



아키텍처 설계

시스템 구조 및 다이어그램



API 명세

인터페이스 정의 (Swagger)



트러블슈팅 가이드

장애 대응 및 해결 절차



회의 노트

주요 논의 및 액션 아이템



FAQ & Onboarding

자주 묻는 질문과 가이드



3. Transparency (투명성)

지식의 사유화를 막고 팀 전체의 자산으로 만드는 공개적 소통 원칙

공개적으로 소통하라



Private DM

개인간의 은밀한 대화



Public Channel

모두가 볼 수 있는 채널

"투명성은 신뢰를 낳고,
신뢰는 속도를 만듭니다."



지식 공유 (Knowledge Sharing)

한 사람의 질문과 답변이 팀 전체의 지식이 됩니다. 정보의 격차를 줄이고 전체의 역량을 높입니다.



팀 학습 (Team Learning)

시니어의 문제 해결 과정을 주니어가 자연스럽게 관찰하고 배울 수 있어, 조직 차원의 성장이 일어납니다.



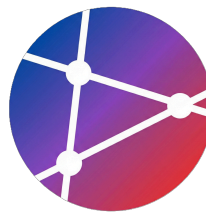
중복 질문 방지

이미 논의된 내용은 검색을 통해 해결할 수 있어, 같은 질문에 반복해서 대답하는 비효율을 제거합니다.



검색 가능성 (Searchability)

히스토리가 남지 않는 휘발성 대화 대신, 언제든지 찾아볼 수 있는 자산이 되어 트러블슈팅 시간을 단축합니다.



4. Over-communicate (과잉 소통)

비동기 환경에서는 정보의 공백이 오해를 낳습니다. 맥락을 충분히 제공하세요.

비동기 소통의 4대 요소



명확하게 (Clear)

애매한 표현을 피하고 의도를 분명히 전달합니다.



자세하게 (Detailed)

필요한 모든 정보를 한 번에 제공하여 핑퐁을 줄입니다.



컨텍스트 포함 (Context)

배경 상황, 시도한 방법, 에러 로그 등을 함께 공유합니다.



예상 응답 시간 (Timeline)

언제까지 피드백이 필요한지 마감 기한을 명시합니다.

✖ Bad Example

"이거 확인 부탁드립니다."

- 무엇을 확인해야 하는지 불명확
- 언제까지 해야 하는지 알 수 없음
- 왜 확인해야 하는지 맥락 부재

✔ Good Example

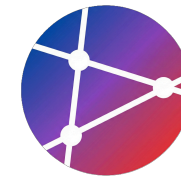
"다음 주 릴리스 전까지 이 PR 리뷰 부탁드립니다."

특히 `auth` 로직 부분에 대한 보안 피드백이 필요합니다."

- 명확한 대상 (PR)과 목적 (리뷰)
- 구체적인 마감 기한 (다음 주 릴리스 전)
- 중점적으로 봐야 할 부분 지정

5) Respect Time Zones (시간대 존중)

글로벌 팀 협업 시 물리적 거리와 시간차를 고려한 소통 방식이 필수적입니다.



🌐 팀 분포 및 시간차 예시

Overlap Time이 거의 없는 상황

🇺🇸 San Francisco (UTC-8)

09:00 AM

🇬🇧 London (UTC+0)

05:00 PM

🇰🇷 Seoul (UTC+9)

02:00 AM (+1)

❌ 나쁜 접근 방식

"지금 회의 가능하세요?"

→ 상대방은 새벽 3시일 수 있음 (수면 방해)

- 즉각적인 응답을 기대하는 태도
- 상대방의 근무 시간을 고려하지 않은 연락
- 자신의 시간대를 기준으로 일정 제안

✅ 좋은 접근 방식

"다음 주 화요일 UTC 14:00 회의 어떠신가요?"

→ UTC 기준 제안 및 비동기적 일정 조율

- "24시간 내 피드백 부탁드립니다" (기한 명시)
- 모두가 깨어있는 'Core Hours' 활용
- 비동기 커뮤니케이션을 기본 원칙으로 설정

비동기 협업 도구

효율적인 협업을 위한 툴킷

GitHub Discussions, Wiki, ADR 등
문서 중심의 비동기 협업을 지원하는
핵심 도구들을 소개합니다.



GitHub Discussions & Wiki

팀의 지식 축적과 열린 소통을 위한 비동기 커뮤니케이션 도구 활용법



GitHub Discussions



Issues

vs



Discussions

실행 가능한 작업
버그, 기능 구현

열린 토론
질문, 아이디어, 공지

RECOMMENDED CATEGORIES



Announcements



Ideas



Q&A



Show and Tell



General



GitHub Wiki

PRIMARY USAGE

- ✓ 프로젝트 문서 및 온보딩 가이드
- ✓ 개발 가이드라인 및 아키텍처 문서
- ✓ API 명세 및 트러블슈팅

WIKI STRUCTURE EXAMPLE

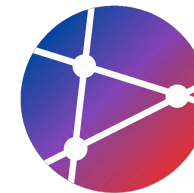


Home

- Getting Started
 - ↳ Development Setup
 - ↳ Coding Standards
- Architecture
 - ↳ System Overview
 - ↳ Database Schema
- Guides

ADR & RFC: 의사결정 문서화

중요한 아키텍처 결정을 기록(ADR)하고, 변경 사항에 대한 팀의 합의를 도출(RFC)하는 표준 프로세스



과거/현재의 기록

Architecture Decision Records (ADR)

"왜 이 기술을 선택했는가?"에 대한 맥락과 결과를 불변의 문서로 남겨 히스토리를 관리합니다.

- **구성:** Status, Context, Decision, Consequences
- **목적:** 지식 전수, 반복 논쟁 방지, 온보딩 가속화

File Structure

```
docs/adr/  
├── 0001-use-jwt-authentication.md  
├── 0002-adopt-microservices.md  
└── template.md
```



미래를 위한 논의

Request for Comments (RFC)

주요 변경사항 도입 전, 설계를 공유하고 피드백을 수집하여 기술적 합의를 도출하는 과정입니다.

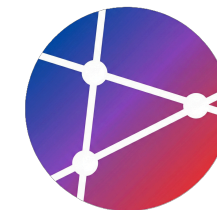
- **과정:** 초안 작성 → Discussion 게시 → 피드백 → 결정
- **항목:** Summary, Motivation, Design, Alternatives

Workflow

Draft → Discuss → Decide → Implement

Summary & Assignments

이번 주차의 핵심 내용을 정리하고 실습 과제를 확인합니다.



실습 과제 (Assignments)

- ✓ **GitHub Discussions:** 카테고리 구성 및 RFC 형식의 첫 Discussion 작성 후 피드백 수집
- ✓ **Wiki 구축:** Getting Started, Guide, Troubleshooting 등 최소 3개 페이지 작성
- ✓ **ADR 작성:** 템플릿을 생성하고 최근 기술 결정 사항을 docs/adr/에 문서화
- ✓ **자동화:** Issue 자동 응답 또는 SLA 추적 워크플로우 1개 이상 구현



핵심 요약 (Key Takeaways)

- **Default to Async:** 동기적 소통은 예외적인 경우에만 사용하고, 기본은 비동기를 지향합니다.
- **Write Things Down:** 모든 결정과 배경을 기록하여 휘발되지 않는 영구적 지식으로 만듭니다.
- **Over-communicate:** 맥락을 포함하여 명확하고 자세하게 소통하여 오해를 줄입니다.
- **Respect Time Zones:** 상대방의 시간대를 고려하고 워크플로우를 자동화하여 효율을 높입니다.



다음 단계 (Next Steps)

다음 주차 강의 주제 및 사전 학습 자료는 GitHub Discussions의 **Announcements** 카테고리를 통해 공지될 예정입니다.

실습 과제 제출 기한은 **다음 수업 전일 23:59**까지이며, PR 링크를 LMS에 제출해주세요.

"The future is asynchronous."

- Remote Work Revolution